Iteration 3 — Change Report

Nine bags, four questions, and one behaviour-tree gate that threw away a run

Summary

Run 8 did not fail because the cost map was wrong or because the robot was in a bad place. It failed because the planner returned **START_OCCUPIED**, and the recovery branch of the behaviour tree is gated on an error-code list that does not contain that code. The round robin — back up, wait, back up, clear — **never ticked once**. Backing up is exactly the right answer to a start cell that has gone inscribed, and the robot was not allowed to try it. That gate is now fixed, and it is the most important change in this iteration.

The decay work does not behave the way the report describes it, and the change made to it bought almost nothing. Raising `decay_acceleration` from 5 to 20 shortened the in-view lifetime from 1.0 s to 0.7 s, measured. Raising `voxel_decay` from 15 to 30 changed nothing measurable at all, and in any case it was only applied to the global cost map — the local one, the map the controller actually reads, was still on 15. That is now matched up.

The oscillation is not a software problem. Every control loop in the stack hit its deadline in every autonomous run. What the bags do show is **280–320 ms of dead time** between a yaw command and the robot responding, and a yaw rate **25% larger than the wheel odometry believes**, consistently, in all five autonomous runs. That is a wheel-geometry calibration error, it lives outside the two files this report is allowed to touch, and it is also the best explanation for the footprint being a centimetre off in RViz.

Finally, the speed: raising `vx_max` to 1.0 could not work, because `prune_distance` was left at 2.5 m against a 3.0 m rollout horizon. That has been raised along with the three path-critic look-aheads.

> **The one-line version**
>
> Fix the behaviour-tree error-code gate before the next run. Everything else in this report is secondary to the fact that a single marginal excursion ends a run with the recovery sequence unused.

1. Is the cost map decaying and clearing the way you want?

Short answer: the timers work exactly as the source says they do, but they are not measuring what the report assumes they are measuring, and the two changes made this iteration had very different value — one small, one zero.

What the code actually does

From `spatio_temporal_voxel_grid.cpp` (STVL 2.6.2, the version installed under `/opt/ros/kilted`), once per update cycle, for every occupied voxel:

```
age   = now − voxel.stamp
base  = voxel_decay − age     (decay_model 0, linear)
accel = (1/6) × decay_acceleration × age³    (inside a clearing frustum only)

inside a frustum:  if (base − accel < 0) erase; else voxel.stamp −= accel
outside:          if (base < 0) erase
```

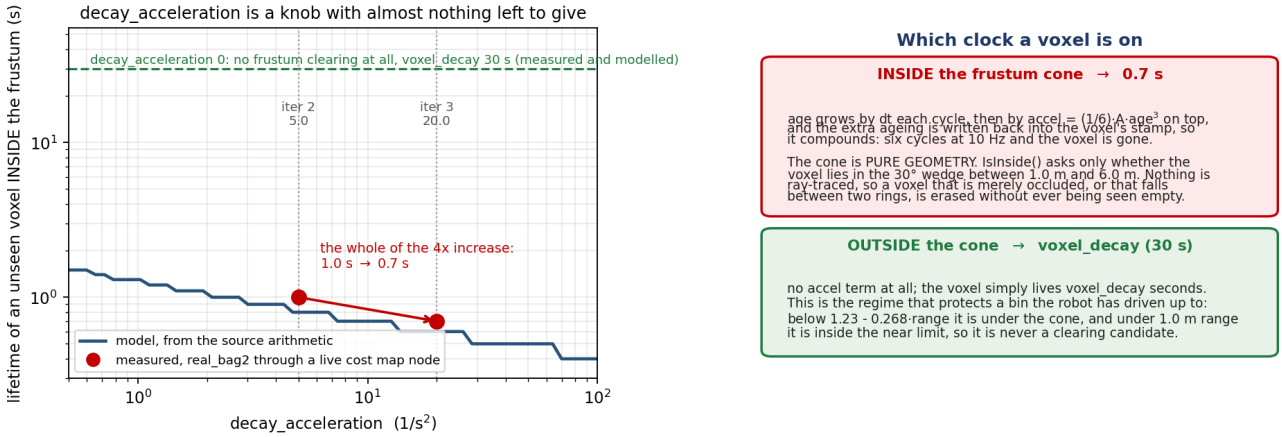
The detail that is easy to miss is `voxel.stamp -= accel`. The extra ageing is written back into the voxel, so the next cycle starts from a larger age, which produces a larger `accel`, and so on. In-frustum clearing is therefore a runaway over a handful of cycles, not the root of `base(age) = accel(age)`. Solving that root for the global map's numbers gives 2.03 s; the actual lifetime is 0.8 s, because of the write-back.

What that works out to, and what was measured

The bags cannot answer this on their own — see the box at the end of this section — so the measurement was made by replaying `real_bag2` into a live `nav2_costmap_2d` node publishing at 10 Hz, and timing how long each cluster's cells outlived the last LiDAR return that landed on them (`claude_tools/decay_replay.py`).

decay_acceleration	model	measured on real_bag2	note
0 (no frustum clearing)	voxel_decay	never cleared in the replay	the voxel_decay timer is the only clock
5.0 (iteration 2)	0.8 s	**1.0 s**	every cluster, identical
20.0 (iteration 3)	0.6 s	**0.7 s**	every cluster, identical

Every cluster in that scene cleared in the same 0.7 s whether `voxel_decay` was set to 15 or to 30. That is the answer to “it felt shorter than 30 seconds”: it was 0.7 s, not 30 s, because almost everything in that scene was still geometrically inside the clearing frustum, and `voxel_decay` only governs voxels *outside* it. The 30 s you set was never reached because it was never consulted.



Left: the in-frustum lifetime against decay_acceleration, from the source arithmetic, with the two measured points. The knob is almost exhausted — a 4× increase bought 0.3 s. Right: the two clocks a voxel can be on, and which one it lands on.

Where it does not match what you asked for

You wrote that the behaviour you want is “if a cell is within view, and we confirm via raytracing that it is no longer occupied, clear it very quickly”. STVL cannot do that, because **STVL never ray-traces**. Its frustum test, `ThreeDimensionalLidarFrustum::IsInside()`, is pure geometry: it asks only whether the voxel lies inside the 30° wedge between `min_z` 1.0 m and `max_z` 6.0 m. It does not ask whether anything was actually observed along that direction. Two consequences follow, and both are real:

Occlusion. A voxel hidden behind a nearer object is inside the cone, so it is on the 0.7 s clock even though the sensor never saw through to it. This is the same class of problem as iteration 2's, arriving by a different route — it is milder, because the robot has to stay still for 0.7 s rather than merely sweep a ray past, but it is the same failure to distinguish “seen empty” from “not seen”.

Ring gaps. The VLP-16's rings are 2° apart, so at 6 m they are 0.21 m apart vertically and at 8 m 0.28 m — against a 0.025 m voxel. A voxel that falls between two rings gets no return and is cleared in 0.7 s. In practice a bin spans several rings and is re-marked every sweep, so it survives; but thin or distant structure will be eroded.

Raising `decay_acceleration` makes both of these slightly worse and buys 0.3 s of responsiveness. It has been left at your value of 20.0, because the measured difference is small either way, but 5.0 was the safer number and there is no evidence that 20 is better.

Why the decay runs could not be measured from the bags, and what was changed

`always_send_full_costmap` was `false`. In `Costmap2DPublisher::publishCostmap`, a full message on `/costmap` is only sent when the resolution, the size or the **origin** changes; otherwise a delta goes to `/costmap_updates`, which is not recorded. Both maps roll, so the origin only moves while the robot does.

A **stationary** robot therefore publishes nothing on `/costmap` at all. Runs 2 and 4 deliberately parked the robot and waited for the decay, which is precisely the interval with no data in it: real_bag2 has 8 local cost map frames spanning 0.1–8.6 s of a 27 s bag, and real_bag4 has a hole from 19 s to 31 s.

It is now `true` in both maps. This does not raise the publish rate — still 2 Hz — it only makes each publish complete, so the next set of bags can be analysed directly.

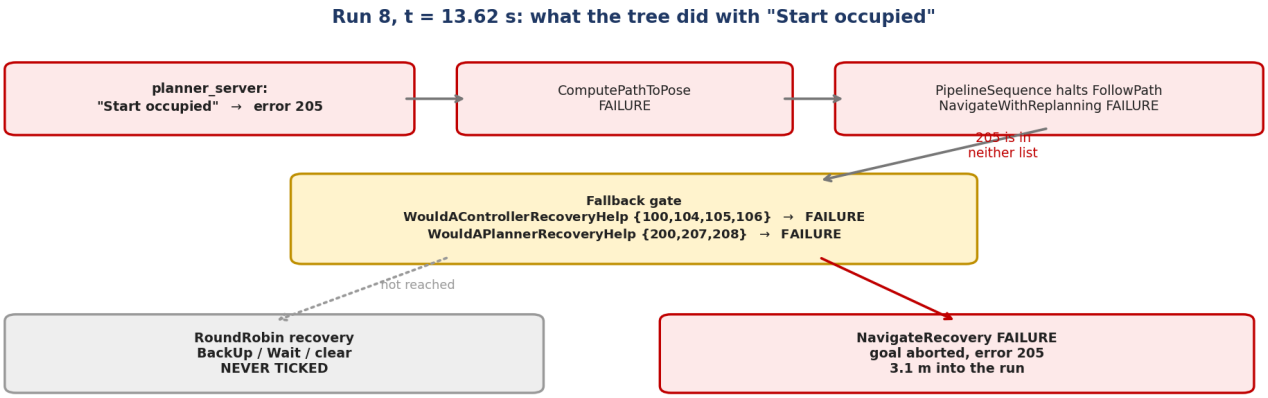
2. Why run 8 failed and run 9 succeeded

The honest answer to the question as posed is: **there was no robust difference between them.** Run 8 crossed a threshold by a few centimetres and run 9 did not, and the reason that was fatal rather than a momentary stumble is the behaviour tree, not the cost map.

What the logs say, verbatim

```
13.33  controller_server  Optimizer fail to compute path
13.61  planner_server     Computing path to goal.
13.62  planner_server     GridBasedplugin failed to plan from (2.14, 1.69)
                               to (1.55, 6.42): "Start occupied"
13.62  compute_path_to_pose  Aborting handle. error_code:205
13.65  BT  ComputePathToPose:           RUNNING → FAILURE
13.65  BT  WouldAControllerRecoveryHelp: IDLE → FAILURE
13.65  BT  WouldAPlannerRecoveryHelp:    IDLE → FAILURE
13.65  BT  Fallback:                   RUNNING → FAILURE
13.65  BT  NavigateRecovery:           IDLE → FAILURE
13.65  bt_navigator      Goal failed error_code:205
```

`ComputePathToPose` error 205 is `START_OCCUPIED`. `WouldAPlannerRecoveryHelp` hard-codes its list in C++ as `{200 UNKNOWN, 207 TIMEOUT, 208 NO_VALID_PATH}`, and `WouldAControllerRecoveryHelp` as `{100, 104, 105, 106}`. 205 is in neither, so the gate returned FAILURE and the whole recovery branch below it was skipped. There is no BackUp, no Wait and no ClearEntireCostmap anywhere in the run-8 behaviour-tree log — the round robin was never entered.



Backing up is exactly the right answer to a start cell that has gone inscribed. The robot was never allowed to try it.

The path run 8 actually took through the tree. Everything below the gate is the recovery sequence tuned in iterations 1 and 2; none of it ran.

What was different about the cost map

Very little, and that is the point. Measuring the distance from the robot's centre to the nearest lethal cell in the published global cost map, across each run (`claude_tools/start_cell.py`):

	closest the centre came to a lethal cell	threshold that triggers START_OCCUPIED
run 8 (failed)	0.50 m, at t = 13.68	0.22 m
run 9 (clean)	0.53 m, at t = 19.90	0.22 m

The 0.22 m is the inscribed radius of the configured footprint — the front edge sits 0.22 m ahead of `base_link`, so that is the nearest footprint edge to the origin, and the inflation layer paints cost 253 within that radius of any lethal cell. `GridCollisionChecker::inCollision` rejects a pose outright when the centre cell is exactly 253 or 254, without even consulting the footprint.

The occupancy that triggered it is **not in the bag**. At the refused pose (2.14, 1.69) the published global cost map reads cost 154, 0.51 m from the nearest lethal cell, in every frame from 11.6 s to 14.9 s — including the frame at 13.68 s, 60 ms after the refusal. The cost map the planner read at 13.62 s and the one that reached the bag therefore disagreed, which means the mark was transient and shorter than one 0.6 s publish interval. The most likely mechanism is STVL's own `update_footprint_enabled: true`, which clears voxels inside the robot's footprint on every update: a mark the robot drives onto is erased within a cycle or two of causing the refusal. That is inference, not measurement, and it is the main reason `always_send_full_costmap` was turned on.

The matching MPPI failure at 13.33 s — “Optimizer fail to compute path” — is consistent with the same transient blob: both the controller and the planner saw something at the robot that was gone by the next publish.

What this means for the next run

With the gate fixed, the same event now runs the round robin: BackUp 0.5 m, wait, BackUp 0.8 m, wait, then the cost map clears as a last resort. Reversing is the correct response to a start cell that has gone inscribed, and it should turn run 8's outcome from an abort into a two-second pause.

It does **not** stop the robot getting that close in the first place. That is the calibration problem in section 3.

3. The oscillation: this one is the machine, not the software

Three measurements, in order of how much they rule out.

The loops are not missing deadlines

The per-message rates printed by a bag summary are misleading here, because they average over the idle stretches before the goal arrives. Measuring the gaps only while each loop is actually running (`claude_tools/loop_timing.py`) gives, in all five autonomous runs:

loop	configured	measured p50	p99	cycles late
controller (cmd_vel out)	10 Hz	100 ms	100–101 ms	0%
MPPI raw command	10 Hz	100 ms	104–112 ms	0%
local cost map update	10 Hz	100 ms	105–107 ms	0%
global cost map update	5 Hz	200 ms	225–244 ms	0%
planner (BT RateController)	2 Hz	551–560 ms	574–591 ms	0%

The budget work from iteration 1 is holding, and STVL did not eat it. The planner runs at 1.8 Hz rather than 2.0 because the tree tick itself is not free; that is not worth chasing.

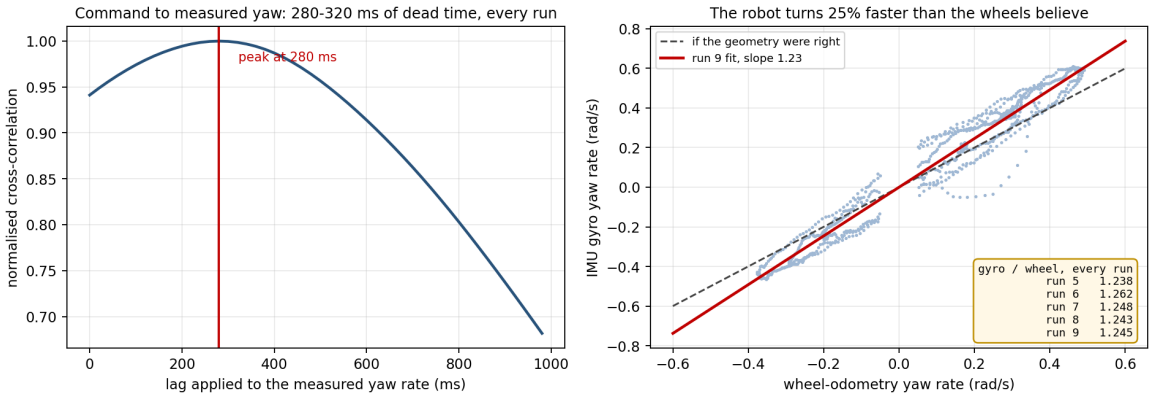
It is not plan churn either

Your hypothesis was that the high planning frequency was changing the local path angle. Measuring how far the plan's heading 1 m ahead of the robot moves between consecutive `/plan` messages:

	run 5	run 6	run 7	run 8	run 9
median	0.7°	2.1°	2.7°	1.3°	3.7°
p90	5.8°	7.6°	8.0°	7.8°	10.1°
max	14.5°	11.5°	19.0°	12.5°	15.7°

A few degrees between replans is not what makes a robot weave. The goal-approach flapping you saw in run 5 is a different thing and is expected: `goal_heading_mode: ALL_DIRECTION` lets the planner pick whichever of the 16 lattice headings is cheapest, so a small cost map change flips the choice. You asked for heading-agnostic goals, that is the price, and it is bounded — GoalCritic takes over inside 1.5 m and `xy_goal_tolerance` is 1.0 m.

What the bags do show



Left: cross-correlation of the commanded yaw rate against the measured one. Right: the IMU against the wheel odometry. The wheel odometry tracks the command (slope 0.86–0.99), and the gyro says the robot turned 25% further than the wheels computed — in every run.

280–320 ms of dead time between a yaw command and the robot responding, agreed on independently by the wheel odometry and the IMU. At 10 Hz that is three control cycles of latency inside a loop that is otherwise perfectly on time. MPPI has no model of it: its rollouts assume the command takes effect now.

A yaw rate 25% larger than the wheel odometry computes. This is the cleanest number in the whole iteration: `gyro / wheel` is 1.238, 1.262, 1.248, 1.243 and 1.245 in runs 5 to 9. It cannot be a controller-tracking problem, because `diff_drive_controller` turns a commanded *wz* into a wheel-speed difference using its configured `wheel_separation` and then computes odometry *wz* back from the wheel speeds with that same number — the constant cancels, and survives only against an independent sensor. The IMU is that sensor.

The parameter, and what it implies

`imprimis_hardware_platform/config/diffbot_controllers.yaml:15`
`wheel_separation: 1.016  # approximation. this is equal to base_length.`

The comment is the bug. The track width has been set to the body length. A ratio of 1.245 implies a true track of about **1.016 / 1.245 = 0.816 m** — 32 inches against the 40 inches configured, which is a plausible real number for this chassis. **Measure it and set it.** This file is outside the two this report may change.

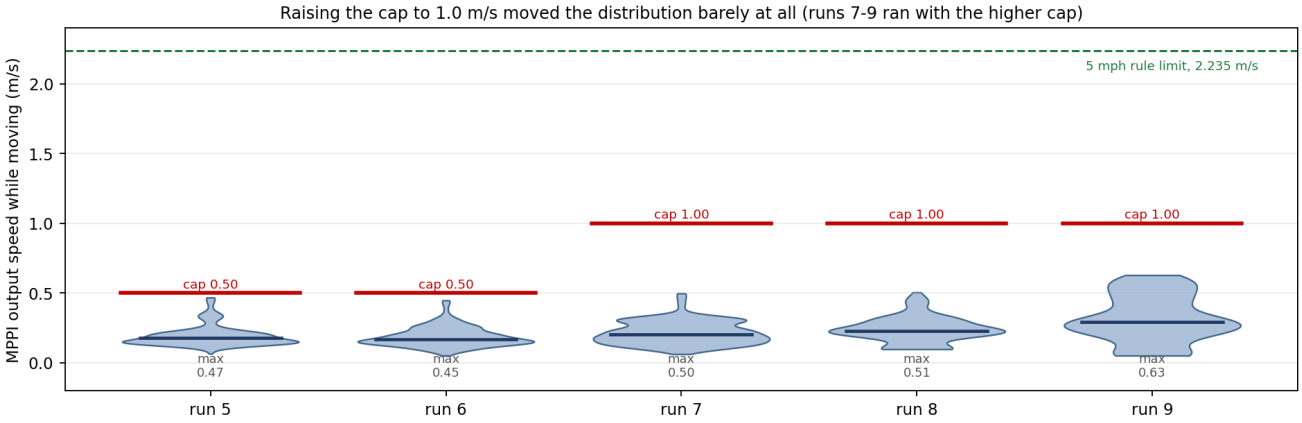
While you are in there: `wheel_radius` is 0.165 in that file and 0.18 in `imprimis_description/urdf/diffbot_description.urdf.xacro:21`, a 9% disagreement. An attempt to pin the linear scale down from the bags, by using the LiDAR as a tape measure, was **inconclusive** (r^2 0.02–0.84 across runs — /velodyne_points_nav has the ground removed and a 0.60 m height floor, so the nearest surface ahead jumps between objects). This one needs a tape measure, not a bag.

A 25% yaw scale error with 300 ms of dead time is a good recipe for overshooting a turn, and overshooting a turn beside a bin is how run 8's start cell went inscribed. Fixing the calibration is likely to do more for both the weaving and the clearance than any further parameter tuning.

And the footprint that looks a centimetre off

Wheel-integrated heading and gyro-integrated heading part company by 12–14° over a 40 s run, and 25° in run 9. The EKF fuses the IMU, so `/odometry/filtered/local` recovers most but not all of that. The tail of the footprint is 0.86 m behind `base_link`, so a residual heading error of only 2° puts the tail 3 cm sideways of where it is drawn; add the 2.5 cm cost map cell and a reading of “3 cm in life, 4 cm in RViz” is exactly what you would expect. Same root cause, same fix.

4. Why raising the speed cap did almost nothing



MPPI's own output while moving. Runs 7 to 9 ran with vx_max and top_speed at 1.0; the distribution hardly moved, and no run put even 1% of its moving samples above 0.8 of the cap.

The provable reason is `prune_distance`. `PathFollowCritic` scores each rollout by the distance from its *last* point to the path point at `furthest_reached_path_point + offset_from_furthest`. That carrot normally sits ahead of every rollout, so reaching further always scores better — but the index is clamped to the end of the pruned path. The rollout horizon is `time_steps 30 × model_dt 0.1 = 3.0 s`, which at 1.0 m/s is 3.0 m of trajectory, against a path pruned at 2.5 m. Past that point a faster rollout ends further *beyond* the carrot and is penalised for it, so 2.5 / 3.0 = 0.83 m/s is a hard disincentive ceiling that no amount of raising `vx_max` can lift.

The three `offset_from_furthest` values are a secondary limit. They count path points, and the Smac lattice emitted 2.8–2.9 cm spacing in these runs (logged per plan), so 24 points was a 0.68 m carrot — sized for 0.5 m/s, and under a quarter of the 3.0 s horizon at 1.0 m/s. They have been scaled with the cap.

Your own observation that raising `vx_std` helped is borne out: run 7 topped out at 0.50 m/s with `vx_std` 0.2 and run 9 reached 0.63 m/s with 0.4. It has been left at your value of 0.4. If the speed is still short after these changes, that is the next knob.

5. What was changed

Only `Course2027.yaml` and `recovery_nav_bt.xml`. The originals are in `iterations/3/OldFiles/`.

recovery_nav_bt.xml

change	from	to	why
Outer recovery gate	WouldAControllerRecoveryHelp + WouldAPlannerRecoveryHelp	Two AreErrorCodesPresent nodes with explicit lists: controller `100;104;105;106;107`, planner `200;205;206;207;208`	Adds 205 START_OCCUPIED, 206 GOAL_OCCUPIED and 107 CONTROLLER_TIMED_OUT. This is the run-8 fix. 201–204 and 101–103 are deliberately left out: those are configuration faults that retrying cannot repair.
Inner FollowPath gate	WouldAControllerRecoveryHelp	AreErrorCodesPresent `100;104;105;106;107`	Same list, so a controller timeout retries once instead of propagating straight to the outer branch.

The round robin itself, the removal of Spin, the BackUp distances and the clear-as-last-resort ordering are all unchanged from iteration 1.

Course2027.yaml

parameter	from	to	why
local_costmap → stvl_layer → voxel_decay	15.0	30.0	The global map was raised to 30.0 this iteration and this one was not, so the two maps disagreed and the forgetful one was the map the controller reads.
always_send_full_costmap (both maps)	false	true	A stationary robot published no cost map at all, which is exactly the interval the decay runs were trying to observe. No change to the publish rate.
prune_distance	2.5	4.0	Must exceed the 3.0 m rollout horizon at vx_max 1.0, or faster rollouts are penalised for running past the end of the pruned path. This is the speed ceiling.
PathFollowCritic.offset_from_furthest	24	45	0.68 m → 1.27 m at the measured 2.8 cm path spacing.
PathFollowCritic.threshold_to_consider	1.5	2.0	Must exceed the look-ahead, or the critic switches off before the carrot is reachable.
PathAlignCritic.offset_from_furthest	20	36	0.57 m → 1.0 m, one second of path at the new cap.
PathAlignCritic.threshold_to_consider	1.2	1.6	Scales with the look-ahead.
PathAngleCritic.offset_from_furthest	20	36	Matched to PathAlignCritic.
PathAngleCritic.threshold_to_consider	1.2	1.6	Scales with the look-ahead.

`decay_acceleration` 20.0 and `vx_std` 0.4, both set by the team this iteration, have been left alone.

How the changes were verified

what	how	result
The behaviour tree parses and every node resolves	Ran a real `bt_navigator` against the edited tree and sent it a goal, so BT.CPP builds the tree. Control: the same test with the node name deliberately corrupted.	Real tree: the only error was “Action server compute_path_to_pose not available”, i.e. it got past parsing and registration. Control: “Error at line 67: Node not recognized: AreErrorCodesBogusXyz”, which proves registration is checked first and the real names passed it.
The config loads	Started a real `controller_server` on the edited `Course2027.yaml` and configured it.	configure → OK. STVL initialised, all seven critics loaded, MPPI configured, no parameter warnings.
The decay numbers	`claude_tools/decay_replay.py` — real_bag2 replayed into a live cost map node at 10 Hz, at four settings.	Reported in section 1. Model and measurement agree within 0.1–0.2 s.

Not verified

The speed changes are reasoned from the critic source and the measured path spacing, not tested in closed loop — that needs the robot or a full simulation. Expect the cruise speed to rise; watch that the robot does not start cutting corners, and be ready to put `prune_distance` back to 3.0 if it does.

The behaviour-tree fix is verified to load and to cover the right codes. Whether BackUp actually frees the robot from an inscribed start cell can only be seen on the course.

6. Recommendations outside the two files

These are flagged, not changed, as agreed.

priority	where	what
1	`imprimis_hardware_platform/config/diffbot_controllers.yaml:15`	`wheel_separation: 1.016` **is wrong by about 25%.** Measure the real track width; the bags imply roughly 0.816 m. This is the single highest-value change available right now — it is behind the weaving, the turn overshoot that put run 8 where it was, and the offset footprint in RViz.
2	same file, line 16, vs `diffbot_description.urdf.xacro:21`	`wheel_radius` is 0.165 in one place and 0.18 in the other. Resolve it by measurement. The bag-based check was inconclusive.
3	the drive chain, wherever the 300 ms lives	280–320 ms of dead time between command and motion, with every loop hitting its deadline. Worth tracing: serial link to the motor controllers, controller-side smoothing, or the EKF. If it cannot be reduced, it should at least be modelled — MPPI assumes its commands take effect immediately.
4	`imprimis_perception` — `surface_tools.classify_lidar`, `surface_max_height: 0.60`	Still open from iteration 2. `/velodyne_points_nav` contains nothing below 0.60 m above the ground, so the cost map only ever learns the top of an object and an obstacle shorter than 0.60 m is never marked at any range.
5	the bag command	`/speed_governor_info` should be `/speed_governor/info` — it recorded zero messages. Add `/local_costmap/costmap_raw` and `/global_costmap/costmap_raw`, which carry the true 0–255 costs instead of the 0–100 rescale, and `/local_costmap/costmap_updates` and `/global_costmap/costmap_updates` as a belt-and-braces alternative to always_send_full_costmap.
6	diagnostic runs only	If you want to catch a sub-second transient like run 8's, raise `publish_frequency` to match `update_frequency` (10 Hz local, 5 Hz global) for that run only. It costs real CPU on a 1200×1200 global grid, so put it back afterwards.

7. What to look for in the next run

1. Does a START_OCCUPIED now produce a BackUp instead of an abort? The behaviour-tree log will say so directly — look for the round robin ticking.
2. Does the cruise speed actually rise, and does the robot start cutting corners if it does?
3. With `always_send_full_costmap` on, the decay can be measured straight from the bag: park the robot beside a bin and watch the cells.
4. If the wheel_separation is corrected before the run, expect the weaving to drop and the clearance beside the bins to improve. That is the one change most likely to matter.